Enabling Data Privacy in Blockchain

Senthilnathan Natarajan
IBM Research

Manish Sethi
IBM Research

Dave Enyeart
IBM

Yacov Manevich
Ava Labs

Artem Barger
TODO

ABSTRACT

Blockchain technology provides a decentralized and immutable ledger, but its inherent transparency—where all data is replicated across all participants—poses a significant barrier to enterprise adoption in domains requiring data privacy. Existing solutions like channels provide data isolation but are coarse-grained, requiring a separate ledger and consensus instance per channel. In this paper, we introduce *private data collections*, a mechanism for managing private data on permissioned blockchains. Our approach decouples private data storage from the replicated ledger: private data is shared only among authorized nodes, while cryptographic hashes are recorded on the ledger for tamper-evidence and serializability verification. A key contribution is a *blind validation* mechanism that enables all nodes to verify the correctness of private transactions using only on-chain hashes, without ever seeing the private data. We design a two-phase gossip-based protocol for proactive dissemination and reactive retrieval of private data, and we formalize the collection abstraction with seven design requirements. We implement our solution within Hyperledger Fabric, leveraging its execute-order-validate-commit architecture, and conduct a performance evaluation demonstrating that private data collections add minimal overhead to transaction throughput while providing fine-grained data privacy. The mechanism has been merged into the Hyperledger Fabric codebase (production-ready since v1.2, with enhancements through v2.5).

PVLDB Reference Format:
Senthilnathan Natarajan, Manish Sethi, Dave Enyeart, Yacov Manevich, and Artem Barger. Enabling Data Privacy in Blockchain. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Enterprise blockchain networks are redefining how organizations collaborate on shared data without a central intermediary. In food safety, Walmart demonstrated that a blockchain-based traceability system can reduce the time to trace contaminated produce from seven days to 2.2 seconds [19, 36]. In pharmaceuticals, the MediLedger network [26] enables drug provenance tracking under the U.S. Drug Supply Chain Security Act, validated through an FDA pilot program. In global trade, GSBN [22] coordinates private shipping documents across dozens of carriers and ports. In mining, MineHub [21] orchestrates supply-chain workflows across thousands of companies. The common foundation underlying these deployments is a permissioned blockchain: a decentralized, immutable, and transparent ledger that establishes trust among mutually distrusting parties without requiring a central authority.

Yet this very transparency creates a fundamental tension. Enterprise data is inherently private—pricing agreements between a manufacturer and supplier, patient medical records governed by HIPAA [17], identity credentials protected under GDPR [10]. Storing such data on a fully replicated ledger exposes it to every network participant, including direct competitors. The consequence is stark: organizations must either accept unacceptable privacy violations, or keep private data in siloed off-chain systems, sacrificing the consistency, auditability, and trust guarantees that motivated the blockchain in the first place. This privacy-transparency tradeoff is widely recognized as one of the primary barriers to enterprise blockchain adoption.

Existing approaches are either too coarse-grained, too expensive, or too fragile. *Channels* [1]—essentially private sub-networks—provide isolation but require a separate ledger, consensus, and management for each data-sharing relationship, an operational burden that scales poorly as the number of bilateral relationships grows. *Zero-knowledge proofs* [2, 23] offer strong cryptographic privacy but impose seconds-per-proof computational costs that are prohibitive for high-throughput enterprise workloads. *Trusted Execution Environments* [7, 34] rely on specialized hardware and carry well-documented side-channel vulnerabilities. Off-chain databases coupled with on-chain hashes break transactional atomicity: a shipment record and its private invoice cannot be committed atomically across a blockchain and an external store without fragile two-phase commit protocols. None of these approaches simultaneously achieve fine-grained privacy, transactional atomicity, smart-contract integration, and low overhead without specialized hardware (Table 1).

Table 1: Comparison of Privacy Mechanisms

Mechanism	Fine Grained	Atomic	SC Access	No HW Dep.
Channels	×	✓	✓	✓
Off-chain DB	✓	×	×	✓
ZKPs	✓	✓	Limited[1]	✓
TEEs (e.g., SGX)	✓	✓	✓	×
Pvt. Collections	✓	✓	✓	✓

This paper presents *private data collections* (PDCs), a mechanism that enables fine-grained data privacy within a single blockchain

[1] ZKP-based smart contracts require expressing all business logic as arithmetic circuits or constraint systems, which limits expressiveness compared to general-purpose languages and imposes significant per-proof computational cost (seconds per proof).

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

ledger. The key insight is that by storing only the cryptographic hash of private data on the replicated ledger while confining actual data to authorized nodes, the entire network can verify the serializability and integrity of private transactions *without ever seeing the private data itself*—a property we call *blind validation*. Authorized peers process private data in plaintext and simultaneously record its hash on the public ledger; at commit time, any node can validate conflicts by comparing hash versions alone, with no access to the underlying private data required. This eliminates the need for expensive cryptographic primitives or specialized hardware. Unlike channels, PDCs operate within a single ledger; unlike ZKP-based approaches, they impose minimal computational overhead; and unlike off-chain stores, they guarantee transactional atomicity across public and private state. However, because private data is excluded from the consensus-driven block delivery, a critical challenge remains: ensuring that all authorized nodes obtain the private data they need before committing a block. We address this with a gossip-based protocol that proactively disseminates private data during transaction simulation and reactively retrieves missing data before commit, tolerating network partitions and dynamic membership changes.

The main contributions of this paper are as follows:

- We formalize the private data collection abstraction, identify seven design requirements (atomicity, hash-data integrity, smart-contract access control, authorized execution, dynamic membership, data lifecycle management, and data availability), and show that the *execute-order-validate-commit* replication protocol is uniquely suited to support them (§3).
- We design a dual-layered storage architecture with a *blind validation* mechanism and formally prove that on-chain hashes serve as a conflict-complete proxy for private state, enabling network-wide serializability verification without exposing private data (§4).
- We propose a two-phase gossip-based protocol combining proactive dissemination with reactive retrieval, supporting dynamic membership changes and tolerating network partitions (§5).
- We implement private data collections in Hyperledger Fabric, an open-source permissioned blockchain platform. The feature has been merged into the Fabric codebase (production-ready since v1.2, with continued enhancements through v2.5) and is deployed in production by organizations including Walmart, MineHub, GSBN, MediLedger, Honeywell, and Oracle.

The remainder of this paper is organized as follows. Section 2 motivates the need for data privacy through real-world use cases. Section 3 presents the core idea of our proposed mechanism and analyzes the suitability of different blockchain replication protocols. Section 4 details the design of private data transaction processing. Section 5 describes the data dissemination and retrieval protocol. Section 6 presents the threat model and security analysis. Section 7 discusses implementation. Section 8 presents a performance evaluation. Section 9 describes real-world adoption and impact. Section 10 discusses related work, and Section 11 concludes.

2 MOTIVATION FOR DATA PRIVACY

Enterprise blockchain networks span diverse domains—supply chains, healthcare, trade finance—where organizations must share data for coordination while protecting proprietary or regulated information. We illustrate the privacy challenge through two representative use cases.

Food Traceability. In 2006, a major *E. coli* outbreak linked to bagged spinach infected 199 people across 26 U.S. states [29]. Regulators took two weeks to trace the contamination source, during which significant quantities of safe food were unnecessarily wasted [14]. A blockchain-based traceability system addresses this by providing a tamper-evident, shared record—Walmart demonstrated that blockchain could reduce backtrace time from seven days to 2.2 seconds [19, 36]. However, recording all transaction details (invoices, pricing, customer lists) on a replicated ledger *leaks proprietary business information* to every participant, including competitors. Food standards authorities [9, 11, 12] mandate detailed record-keeping, yet much of this data is commercially sensitive. Organizations face a dilemma: accept privacy violations, or keep sensitive data off-chain and sacrifice the consistency and auditability that motivated blockchain adoption.

Personal Data Management. In healthcare, patients see an average of 18.7 different practitioners in their lifetime [13], yet Electronic Medical Records remain siloed in hospital-centric systems. A blockchain solution could give patients ownership of their records and enable instant verification by third parties (insurers, pharmacies). But directly storing medical diagnoses or treatment histories on a replicated ledger is unacceptable—legal mandates like the HIPAA Privacy Rule [17] require providers to protect patient confidentiality, and regulations such as the GDPR [10] impose strict controls on personal data processing.

These use cases highlight a common pattern: blockchain provides the trust and coordination benefits enterprises need, but its inherent transparency is incompatible with the fine-grained data privacy that real-world deployments require. This motivates our work on private data collections, which we introduce in the following section.

3 PROPOSING PRIVATE DATA COLLECTIONS FOR DATA PRIVACY

A naive approach to achieving data privacy involves using an external, off-chain data store (e.g., a traditional database) to house private data while only storing its cryptographic hash on the blockchain. However, this decoupled approach is insufficient for enterprise applications as it breaks the fundamental trust and consistency models of the blockchain. First, it cannot guarantee transactional atomicity: in a food supply chain, an update to a public shipment record and its private invoice must be atomic, yet achieving atomicity via two-phase commit across a decentralized blockchain and an external database is notoriously fragile. Second, it lacks integrated smart-contract access control: an integrated solution allows the same smart contract that enforces business logic to also gate access to the private data—ensuring, for example, that private medical records are only accessible to authorized practitioners. Finally, dynamic membership becomes a manual, error-prone process: if a new organization joins a consortium, they must be manually granted access to historical external data, whereas an integrated solution

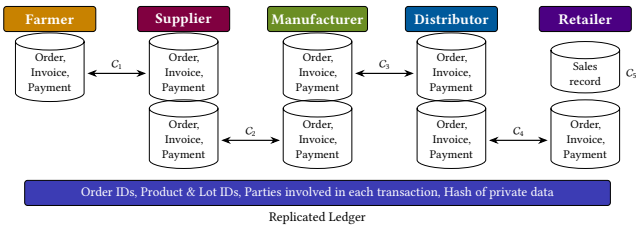

Figure 1: The private data is visible only to the appropriate parties and the proof of data existence is stored in a replicated ledger along with necessary information needed to enable traceability.

allows blockchain nodes to automatically synchronize private state according to the collection policy. By managing private data directly within the blockchain nodes, we ensure that the private data inherits the same availability, consistency, and security properties as the replicated ledger. To achieve this, we propose an integrated *private data collection* mechanism.

3.1 Key Idea

Our proposal achieves data privacy by decoupling the dissemination and storage of private data from the maintenance of the shared, immutable transaction ledger. To hide private data from specific members in a blockchain network, we propose two complementary mechanisms: (1) *decoupled storage*, where private data is shared and stored only on nodes owned by authorized members via a private data *collection* with defined membership; and (2) *on-chain evidence*, where the cryptographic hash of the private data is stored on all nodes as part of the replicated ledger for tamper detection.

Formally, a private data collection C is defined as a tuple:

$$C = (id, \mathcal{M}, \mathcal{E}, n_{dissem}, n_{required}, \tau)$$

where id is a unique collection identifier, $\mathcal{M}$ is the membership policy specifying the set of authorized organizations, $\mathcal{E}$ is the endorsement policy governing write access, n_{dissem} is the maximum number of member organizations targeted during proactive dissemination, $n_{required}$ is the minimum number of member organizations that must acknowledge receipt before a transaction can proceed, and τ is the block-to-live (BTL) retention parameter specifying the number of blocks for which private data remains in the private data store after being committed (where $\tau = 0$ signifies indefinite retention). A smart contract may define multiple collections, each with independent policies. Note that the collection's endorsement policy $\mathcal{E}$ may differ from the smart contract's own endorsement policy, which governs writes to the public state shared across all members. This is because the collection membership $\mathcal{M}$ is typically a subset of the network participants, requiring a distinct endorsement policy drawn from its authorized members.

Figure 1 showcases the use of private data collections in the food traceability use-case. Five collections (C_1–C_5) are defined based on participant relationships: C_1–C_4 each shared between two adjacent parties in the supply chain, and C_5 private to the Retailer. The replicated ledger contains public data (e.g., order IDs, product lots) and the hash of each private data element, while private data (e.g., invoices, payments) are stored in the appropriate collection. This ensures that competitors can verify product provenance without gaining access to private business contracts. The same mechanism extends to personal data management: a hospital stores a patient's medical records in a collection where its node is a member, and when needed—such as for an insurance claim—the record can be copied to a collection where the insurance provider is a member. Because the record's hash is present in the replicated ledger, the receiving party can independently verify its authenticity, resolving the conflict between ledger transparency and patient privacy.

3.2 Design Requirements

The use cases in §2 and the architectural constraints of our key idea give rise to seven design requirements, organized into three categories.

Correctness. A private data mechanism must preserve the transactional guarantees of the underlying blockchain. In the food supply chain, updating a public shipment record and its private invoice must be atomic—a partial update would leave the ledger in an inconsistent state. Similarly, the on-chain hash must faithfully reflect the off-chain data so that any party can detect tampering.

R1 *Transactional Atomicity and Isolation:* When a transaction modifies both public and private data, the system must guarantee atomicity and serializable isolation to maintain overall consistency.

R2 *Hash-Data Integrity:* The hash of the private data stored on the replicated ledger must always remain in sync with the actual private data stored in the off-chain collection, acting as a verifiable proof of existence.

Access Control. Privacy is only meaningful if the system enforces who can read and write private data. In healthcare, only authorized practitioners should access patient records; the same smart contract that enforces clinical business logic should also gate access to the underlying data. Furthermore, private data must never be processed on unauthorized nodes, even transiently during execution.

R3 *Smart-Contract Controlled Access:* Private data in a collection must only be accessible or modifiable via a smart-contract, ensuring all data interactions adhere to agreed-upon business processes.

R4 *Authorized Execution Only:* Transactions involving private data must only be executed on nodes that are members of the specified collections, preventing unauthorized nodes from seeing or processing private data.

Operational. Enterprise consortia evolve over time: new organizations join, existing members leave, and regulations impose data retention limits. In global trade, a new carrier joining GSBN must synchronize historical shipping documents; under GDPR, a patient may exercise the "right to forget," requiring purging of personal data. Because private data is excluded from the consensus-driven block delivery, the system must also guarantee that authorized nodes can obtain it through an alternative mechanism.

R5 *Dynamic Membership:* Collection membership must be mutable. New members must be able to synchronize historical private data, while removed members should lose access to any data persisted after their removal.

R6 *Data Lifecycle Management:* The system must support a block-to-live (BTL) property for private data, allowing for automatic

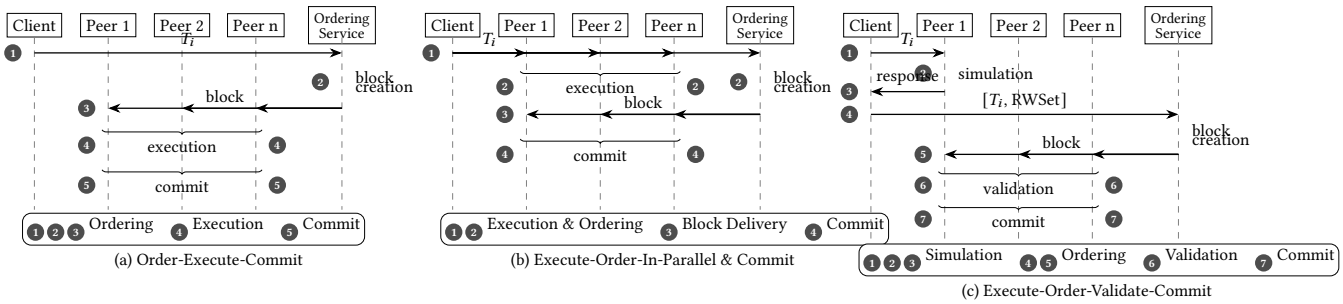

Figure 2: Blockchain replication protocols.

purging to comply with data retention policies or "right to forget" regulations.

R7 *Data Availability:* Since private data is excluded from the consensus-driven block delivery, the system must ensure that all authorized nodes can obtain the private data they need before committing a block, even under network partitions and peer failures.

3.3 Implication of Blockchain Replication Protocols

To realize the requirements outlined in §3, we must identify a blockchain architecture that natively supports decoupled execution and selective data dissemination. In a permissioned blockchain network, a *client* submits transactions on behalf of an application; a *peer* is a node owned by a member organization that maintains a copy of the ledger, executes smart contracts, and validates transactions; and an *ordering service* is a decentralized cluster of nodes—contributed by the same participating organizations—that establishes the total order of transactions via a consensus protocol. Each peer belongs to exactly one organization, and organizations may operate multiple peers for redundancy. We evaluate three prevalent replication protocols, depicted in Figure 2, to determine their suitability as a foundation for our mechanism. Throughout this analysis, we assume a crash-fault tolerant (CFT) model for authorized peers; the full threat model, including Byzantine considerations for the ordering service, is presented in §6.

Order-Execute-Commit and Execute-Order-In-Parallel & Commit. These two protocols—shown in Figures 2(a) and (b)—differ in the relationship between ordering and execution: the first orders before execution, while the second performs both in parallel. However, they share a fundamental limitation: every node must execute (or re-execute) every transaction to maintain ledger consistency. The order-execute-commit protocol (Ethereum [6], Quorum [27], Hyperledger Besu [20]) orders first, then broadcasts blocks for serial execution on all nodes. The execute-order-in-parallel protocol (BlockchainRDBMS [28]) sends each transaction to all peers and the ordering service simultaneously—peers execute while the ordering service establishes the total order in parallel—but still requires all nodes to verify the final block. In both cases, restricting execution to a subset of nodes would break the state machine replication (SMR) guarantees, violating R4 (Authorized Execution Only). Since all nodes execute every smart contract, private data accessed during execution is exposed to the entire network (violating R3), and the absence of a separate validation phase means there is no way to verify private transactions without re-executing them (violating R7).

Execute-Order-Validate-Commit. Figure 2(c) presents four phases: simulation (steps 1–2), ordering (steps 3–5), validation (step 6), and commit (step 7). A transaction is speculatively executed on a subset of nodes according to a specific endorsement policy; the results (read-write sets) are then ordered by a consensus service; finally, all nodes perform validation—including endorsement policy checks and optimistic concurrency control (OCC) [24] based serializability checks on read-write set versions—before committing valid transactions. Hyperledger Fabric [1] implements this protocol. This architecture is uniquely suited as a foundation for our mechanism because its structural properties can be extended to satisfy all seven requirements: since transactions are simulated on a subset of nodes, we can restrict simulation to authorized collection members, enabling R4 and R3; the separate validation phase can be augmented to verify hashes of private data on the replicated ledger, enabling R2; the existing OCC-based validation can be extended to cover both public and private state, enabling R1; the decoupling of simulation from consensus creates a natural insertion point for a peer-to-peer dissemination layer, enabling R5 and R7; and because the architecture already separates execution results from the immutable ledger, private data can be stored independently, enabling lifecycle management through purging while retaining on-chain hashes (R6).

Based on this analysis, we adopt the execute-order-validate-commit protocol as the foundation for our design. The subsequent sections describe the generic design of our private data collection mechanism.

4 DESIGN OF PRIVATE DATA TRANSACTION PROCESSING

To achieve data privacy while maintaining the integrity and consistency of the blockchain, we propose an integrated architecture based on the *execute-order-validate-commit* paradigm. Our design employs a *dual-layered storage strategy* and *partitioned state management* to isolate private data while ensuring global verifiability. We assume a key-value data model for both public and private state, where each state entry is a $(key, value)$ pair identified by a unique key within a namespace, and each key carries a version that is updated whenever the key is written by a committed transaction.

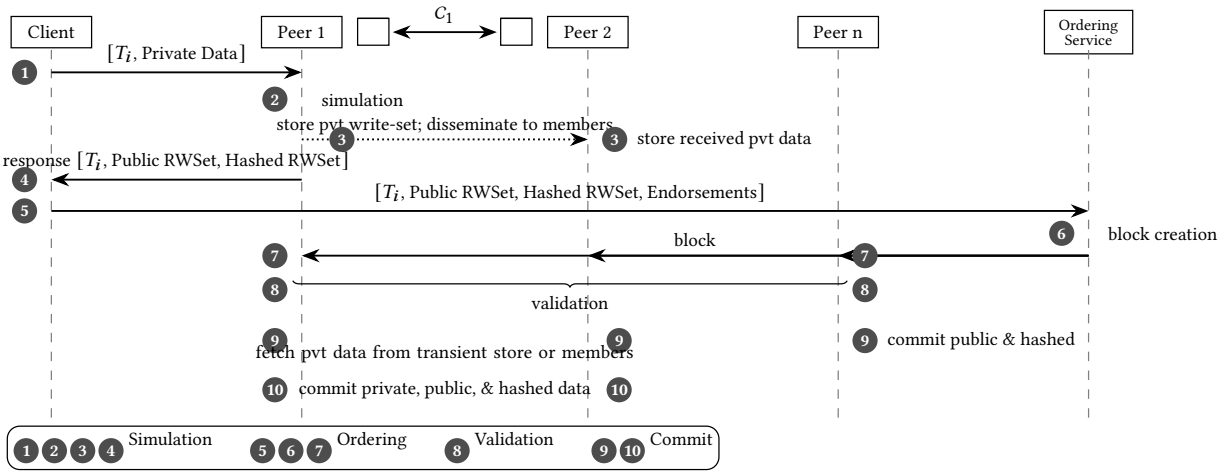

Figure 3: End-to-end transaction flow with private data collections. Steps 1–4: simulation (including private data dissemination), 5–7: ordering, 8: validation, 9–10: commit (including reactive retrieval).

Figure 3 provides an end-to-end overview of the private data collection transaction flow, with numbered steps spanning all four phases. In this figure, Peer 1 and Peer 2 are members of collection C_1, while Peer n is not a member. The subsequent subsections detail the storage architecture and each phase.

4.1 Partitioned State Management

A key challenge in this architecture is integrating private data into the ledger without exposing private data to unauthorized nodes during validation. We solve this by partitioning the state database into three logical domains. Each smart contract is allocated a dedicated *namespace* within these domains, and the access and modification of data within a namespace are governed by an endorsement policy specifying the set of authorized nodes whose signatures are required to validate state changes. The three logical spaces for each namespace are:

- *Public State:* Accessible to all nodes; contains the standard replicated ledger state.
- *Hashed State:* Replicated across all nodes; contains cryptographic hashes of private keys and values, providing the evidence required for tamper-detection and serializability checks.
- *Private State:* Isolated to authorized nodes; contains the actual private key-value pairs. Modifications are governed by the collection's own endorsement policy $\mathcal{E}$, which supersedes the smart contract's default endorsement policy for private data operations.

A key consequence of this partitioning is that Public State and Hashed State are consistent across all peers in the network, providing a globally uniform view of the ledger. Private State, however, varies from peer to peer depending on which collections each peer's organization is a member of—a peer only maintains private state for collections in which it is authorized. This partitioning satisfies **R3** (Smart-Contract Controlled Access) by confining all private data interactions to smart-contract namespaces, and reinforces **R2** (Hash-Data Integrity) by maintaining a replicated Hashed State that mirrors the private state's structure.

4.2 Transaction Simulation

The simulation phase (steps 1–4 in Figure 3) begins when a client submits a transaction proposal to a subset of peers that are members of the relevant collections, satisfying **R4** (Authorized Execution Only) by ensuring that private data is only processed on authorized nodes. The client provides private data through a *transient data* field in the proposal. This transient data is used by the smart contract during execution but is excluded from the final transaction payload, preventing private data from being broadcast to the entire network during ordering.

Upon receiving the proposal, each endorsing peer executes the smart contract, which interacts with public and private state through the APIs defined in Table 2. Public data operations (`GetState`, `PutState`, `DelState`) access the standard replicated state, while private data operations (`GetPrivateData`, `PutPrivateData`, `DelPrivateData`) access collection-specific private state. A single transaction may read and write both public and private state within the same smart-contract invocation; however, doing so requires that the endorsing peers satisfy both the smart contract's endorsement policy (for public writes) and the collection's endorsement policy $\mathcal{E}$ (for private writes). Crucially, each private data write API performs dual-recording: for example, `PutPrivateData`(c, k, v) writes (k, v) to the private write-set for collection c and simultaneously records $(H(k), H(v))$ in the hashed write-set, satisfying **R2** (Hash-Data Integrity) by ensuring that every update to private state is inextricably linked to the verifiable replicated ledger.

On successful execution, the peer constructs three types of read-write sets: (1) a *public read-write set* capturing reads and writes to the replicated ledger state, (2) a *hashed read-write set* containing only cryptographic hashes of private keys and values, and (3) one or more *private write-sets*, one per collection involved in the transaction. The example in Table 2 illustrates how a sequence of API calls produces these partitioned outputs. The endorsing peer signs the public and hashed read-write sets as part of its endorsement response and returns it to the client, while the private write-sets are stored in the peer's transient store and proactively disseminated to other collection members as described in §5.

Table 2: Smart-contract APIs for public and private data operations, with examples illustrating simulation and blind validation.

API	*State Effect*	*Read-/Write-Set Effect*
`GetState`(k)	Returns value of k from public state	Records (k, version) in public read-set
`PutState`(k, v)	—	Records (k, v) in public write-set
`DelState`(k)	—	Records delete of k in public write-set
`GetPrivateData`(c, k)	Returns value of k from collection c	Records ($H(k)$, version) in hashed read-set
`GetPrivateDataHash`(c, k)	Returns $H(v)$ for key k in collection c	Records ($H(k)$, version) in hashed read-set
`PutPrivateData`(c, k, v)	—	Records (k, v) in private write-set; ($H(k)$, $H(v)$) in hashed write-set
`DelPrivateData`(c, k)	—	Records delete of k in private write-set; delete marker in hashed write-set
`PurgePrivateData`(c, k)	Permanently removes k from local storage	Like `Del`, but purges history to comply with "right to forget" policies

Simulation Example. Transaction T_i executes: `GetState`(k_1) → `PutState`(k_1, v_2) → `GetPrivateData`(c_1, k_2) → `GetPrivateData`(c_2, k_3) → `PutPrivateData`(c_1, k_2, v_3) → `PutPrivateData`(c_2, k_3, v_4). This produces three partitioned outputs: **Public RW-Set:** R−$\{(k_1, \text{v1})\}$, W−$\{(k_1, v_2)\}$. **Hashed RW-Set:** R−$\{(c_1, H(k_2), \text{v5}), (c_2, H(k_3), \text{v5})\}$; W−$\{(c_1, H(k_2), H(v_3)), (c_2, H(k_3), H(v_4))\}$. **Private Write-Sets:** c_1: W−$\{(k_2, v_3)\}$; c_2: W−$\{(k_3, v_4)\}$. Here, v1 and v5 denote the version read during simulation.

Validation Example (Blind Validation). Before T_i reaches validation, another transaction T_j commits and updates key k_2 in collection c_1, advancing its hashed state version from v5 to v8. This means T_i's read of k_2 (recorded at version v5 via `GetPrivateData`) is now stale. At validation, the validator—which may not be a member of c_1 and has no access to the private data—performs OCC checks using only the public and hashed state: for k_1, it compares the read version v1 against the current public state version (*match* → pass); for $H(k_2)$ in c_1, it compares the read version v5 against the current hashed state version v8 (*mismatch* → **stale read, conflict detected**); for $H(k_3)$ in c_2, v5 still matches (*match* → pass). Transaction T_i is marked invalid due to the stale read on $H(k_2)$, despite the validator never seeing k_2's actual value—this is blind validation.

Because the final order of transactions is determined after simulation, there is a critical temporal gap during which a private write-set may become orphaned if its transaction is later invalidated. Directly writing private data to the permanent store would violate the isolation guarantee, as invalid transactions could corrupt committed state. To address this, nodes utilize a transient store to persist private write-sets, isolating uncommitted private state from the permanent ledger, which is essential for **R1** (Transactional Atomicity and Isolation). Since multiple transactions may be simulated concurrently, the transient store uses the transaction identifier (*txid*) as the primary key to prevent collisions. Data remains in the transient store until the transaction is successfully ordered and reaches the commit phase.

4.3 Transaction Validation and Commit

Once the client collects sufficient endorsements, it submits the transaction (containing the public and hashed read-write sets, but not the private write-sets) to the ordering service, which batches transactions into blocks and delivers them to all peers (steps 5–7 in Figure 3). This ordering phase is unchanged from the base protocol—the ordering service never sees private data.

The validation (step 8) and commit (steps 9–10) phases ensure that only correctly endorsed and serializable transactions are permanently recorded. While endorsement policy evaluation for transactions within a block can be performed in parallel, OCC validation proceeds sequentially in the committed order because the outcome of one transaction (valid or invalid) can affect the state versions seen by subsequent transactions in the same block; alternatively, a serialization graph can be used to validate non-conflicting transactions in parallel [?]. Before a transaction is committed, nodes verify that it has been endorsed according to the smart contract's endorsement policy for public state changes and the collection's endorsement policy $\mathcal{E}$ for private state changes, satisfying **R4** (Authorized Execution Only) and preventing unauthorized entities from recording fraudulent data or hashes on the replicated ledger. The system then performs OCC checks using both the public read-write set and the hashed read-write set: for public state, the validator compares the read versions in the transaction against the current Public State; for private state, the validator compares the hash versions in the transaction against the current Hashed State. A critical property of this design is that the Hashed State serves as a perfect proxy for the Private State for version tracking. Since every update to a private key k is accompanied by a mandatory update to its hash $H(k)$ in the replicated ledger, the version identifiers in the Hashed State are sufficient to detect all write-write and read-write conflicts on private data. This "blind validation" allows the entire network to verify the serializability of both public and private state changes without ever seeing the private data itself. The validation example in Table 2 illustrates this concretely: a stale read on private key k_2 in collection c_1 is detected purely through hashed state version comparison, even by a validator that has no access to the private data.

We now formalize the correctness of blind validation. We adopt the standard OCC serializability criterion [24]: a transaction T_i is *serializable* with respect to a committed history if no key in T_i's read-set has been modified by any transaction T_j that committed after T_i began execution (i.e., the read-set versions at simulation time match the current committed versions at validation time). Let $\mathcal{S}_P$ denote the Private State and $\mathcal{S}_H$ denote the Hashed State. Let $\text{ver}(s, k)$ denote the version of key k in state s.

Theorem 1 (Conflict-Complete Proxy). *The Hashed State $\mathcal{S}_H$ is a conflict-complete proxy for the Private State $\mathcal{S}_P$: for any transaction T_i involving private data, an OCC conflict exists in $\mathcal{S}_P$ if and only if a corresponding conflict exists in $\mathcal{S}_H$. Formally, for every private key k in collection c:*

$$\text{ver}(\mathcal{S}_P, k) \neq \text{ver}_{\text{sim}}(T_i, k) \iff \text{ver}(\mathcal{S}_H, H(k)) \neq \text{ver}_{\text{sim}}(T_i, H(k))$$

Proof. We establish three invariants maintained by the API (Table 2):

(1) *Write Bijection:* By construction, every `PutPrivateData`(c, k, v) atomically produces both a private write-set entry (k, v)

and a hashed write-set entry $(H(k), H(v))$. The commit procedure assigns both entries the same version identifier (block number, transaction index). `DelPrivateData` and `PurgePrivateData` similarly produce corresponding hashed entries. Thus, $\text{ver}(\mathcal{S}_P, k)$ and $\text{ver}(\mathcal{S}_H, H(k))$ are updated in lockstep: $\text{ver}(\mathcal{S}_P, k) = \text{ver}(\mathcal{S}_H, H(k))$ after every committed transaction.

(2) *Read Recording:* Every `GetPrivateData`(c, k) records $\text{ver}(\mathcal{S}_H, H(k))$ in the hashed read-set. Since Invariant 1 guarantees $\text{ver}(\mathcal{S}_P, k) = \text{ver}(\mathcal{S}_H, H(k))$, this recorded version is identical to the version that would appear in a hypothetical private read-set.

(3) *Conflict Detection:* During OCC validation, a conflict on private key k exists iff $\text{ver}(\mathcal{S}_P, k)$ has changed since simulation. By Invariant 1, $\text{ver}(\mathcal{S}_P, k)$ changes iff $\text{ver}(\mathcal{S}_H, H(k))$ changes. By Invariant 2, the validator compares $\text{ver}(\mathcal{S}_H, H(k))$ against the same baseline. Therefore, the OCC check on $\mathcal{S}_H$ detects a conflict iff the corresponding check on $\mathcal{S}_P$ would.

Since both directions hold, blind validation produces neither false negatives (missed conflicts) nor false positives (spurious invalidations). □

Edge Case: Concurrent Membership Changes. A collection's membership policy $\mathcal{M}$ may be updated in the same block that contains private data transactions for that collection. The validator processes transactions within a block either sequentially in their committed order or using a serialization graph to validate non-conflicting transactions in parallel [**?**]: in both cases, a membership change at transaction index j takes effect for all subsequent transactions $j' > j$ in the same block. This within-block ordering is a deliberate design choice—it avoids the complexity of batch-level policy resolution and ensures deterministic validation across all peers. The result is that the endorsement policy and membership checks applied during validation always reflect the correct policy version. A newly added member will not yet have historical private data at validation time; the reactive retrieval and reconciliation mechanisms (§5) handle this asynchronously without affecting the validity of the current block's transactions.

Together, Theorem 1 and the sequential policy application ensure **R1** (Transactional Atomicity and Isolation) across both public and private state.

Once a transaction is successfully validated, the private write-sets are moved from the transient store to the permanent private data store. Because the Private State within a smart contract's namespace reflects only the latest value for each key—overwritten by each subsequent committed transaction—we introduce a separate
textitprivate data store that retains all historical private write-sets indexed by block number and transaction index. This separation ensures that when a new member joins a collection, the complete history of private data can be retrieved and provided to that member.

Handling Missing Private Data. A peer may not possess all private write-sets at commit time. During commit, each peer classifies missing private data into two categories: (1) *eligible missing data*—private data for collections in which the peer's organization is a member but the data has not yet been received (e.g., due to network delays during dissemination), and (2) *ineligible missing data*—private data for collections in which the peer's organization is not currently a member. The peer records both categories along with the block number, transaction index, collection identifier, and the corresponding hashed key-value pairs from the hashed write-set—the latter enabling the peer to verify the authenticity of private data received during retrieval. Eligible missing entries are fetched via the reactive retrieval protocol described in §5, satisfying **R7** (Data Availability). Ineligible entries are retained so that if the organization later becomes a member of the collection, it can identify and synchronize historical private data that it is now authorized to access, supporting **R5** (Dynamic Membership).

Data Lifecycle Management. The private data store supports data lifecycle management through a block-indexed retention policy, satisfying **R6** (Data Lifecycle Management). To comply with privacy regulations like the "Right to Forget," a background process periodically purges expired private data while maintaining the on-chain hashes to ensure the historical audit trail remains tamper-evident.[2]

The design described in this section assumes that private write-sets are available on authorized nodes when needed for commit. In the next section, we address the critical question of *how* this data is disseminated across the network and how missing data is recovered, which is essential for **R5** (Dynamic Membership) and **R7** (Data Availability).

5 PRIVATE DATA DISSEMINATION AND RETRIEVAL

Since private data is excluded from the consensus-driven block delivery, we utilize a specialized peer-to-peer protocol to ensure data availability and fault tolerance. The protocol operates in two complementary phases: proactive dissemination during simulation, and reactive retrieval during commit.

5.1 Proactive Dissemination

During the simulation phase, each endorsing node stores the generated private write-sets in its local transient store and proactively pushes them to a subset of authorized remote peers via a peer-to-peer protocol. For each collection involved in the transaction, the endorser sends the entire collection's private write-set as a single message to eligible peers. The set of target peers is determined by evaluating the collection's membership policy $\mathcal{M}$ against the current view of active peers in the network. The number of peers targeted is governed by n_{dissem}, and the endorser waits until at least $n_{required}$ peers acknowledge receipt before returning the signed endorsement response to the client. This eager strategy ensures that the private data is already present on most authorized nodes by the time the corresponding block is received, minimizing the need for reactive pulling during the commit phase. If the endorser cannot obtain $n_{required}$ acknowledgments before the timeout, it returns a dissemination failure to the client; the client may then retry with a different set of endorsers or abort the transaction. The endorser itself does not re-attempt dissemination, as the client is responsible

[2]While on-chain hashes that remain after purging could, under certain legal interpretations of regulations such as GDPR, still be considered references to personal data, the hash is a one-way function and the remaining hash alone does not reveal any private content. Organizations should seek legal counsel on the applicability of specific data protection regulations to their deployment.

for orchestrating retries—this keeps the endorser stateless with respect to dissemination attempts. Algorithm 1 summarizes the protocol.

Algorithm 1 Proactive dissemination protocol executed during the simulation phase. The endorser pushes each collection's full private write-set to authorized peers before returning the endorsement.

Proactive Dissemination Protocol

Input: Transaction tx with private write-sets

At Endorsing Node e (after simulation):

1. Store all private write-sets for tx in local transient store
2. **for each** collection c in tx's private write-sets **do**
   - $W_c \leftarrow$ collection c's private write-set
   - $P \leftarrow \text{SelectAuthorizedPeers}(c.\mathcal{M}, c.n_{dissem})$
   - **for each** peer $p_i \in P$ **do**
     - Push $(tx.id, c.id, W_c)$ to p_i via gossip
   - Wait until $|\{p_i : \text{ACK received}\}| \geq c.n_{required}$ or timeout expires
   - **if** timeout **then** return dissemination failure
3. Return signed endorsement response to client

At Receiving Peer p_i:

1. Verify sender e is authorized for collection c
2. Verify ledger height of tx is within acceptable range
3. Store $(tx.id, c.id, W_c)$ in local transient store
4. Send ACK to endorser e

5.2 Reactive Retrieval

Prior to the commit of a block, each peer analyzes the transactions and determines if it is missing authorized private data. For each valid transaction, the peer checks every collection referenced in the hashed write-set: if the peer's organization is a member of the collection and the corresponding private write-set is not already present in the local transient store, the collection is marked as missing.

The retrieval process, described in Algorithm 2, employs a prioritized pull strategy. When a peer identifies missing data, it constructs a digest for each missing entry—identified by the tuple (transaction ID, namespace, collection, block number, transaction index)—and first attempts to fetch the full collection-level private write-set from the peers that endorsed the transaction, as they are guaranteed to have possessed the data during simulation. If those nodes are unreachable, the requester broadcasts requests to other authorized collection members via the gossip network. Upon receiving a response containing the private write-set, the requester validates it by hashing the data and comparing the result to the hash recorded in the block, ensuring data integrity even when fetching from potentially untrusted neighbors. Retrieval attempts use a fixed-interval retry bounded by a configurable time threshold to avoid overwhelming the network during periods of high congestion. If retrieval still fails after the retry threshold, the peer records the entries as textiteligible missing data and proceeds with the block commit; a background reconciliation service periodically retries fetching these entries using the same protocol, ensuring eventual data availability without blocking the commit pipeline.

Note that if a smart contract attempts to read private data that is missing from the peer's local store (e.g., due to incomplete dissemination or reconciliation), the texttttGetPrivateData API returns an error, causing the simulation to fail. This fail-fast behavior prevents the endorser from producing a read-write set based on incomplete state, ensuring that only fully informed simulations contribute to transaction endorsements.

Algorithm 2 Reactive retrieval protocol executed before the commit phase. Missing private write-sets are identified per collection; endorsers are prioritized as data sources with gossip as a fallback.

Reactive Retrieval Protocol

Input: Block B with hashed write-sets, local transient store

At Committing Peer p (before commit):

1. **for each** transaction $tx \in B$ **do**
   - **if** tx fails endorsement policy **then skip**
2. **for each** collection c referenced in tx's hashed write-set **do**
   - **if** $p \in c.\mathcal{M}$ **and** c's private write-set $\notin$ transient store
     - Add digest $(tx.id, c.ns, c.id, B.num, tx.idx)$ to MissingSet
3. **if** MissingSet $= \emptyset$ **then** proceed to commit
4. $E \leftarrow$ endorsers of tx *// priority targets*
5. Send pull request with MissingSet digests to E
6. **for each** private write-set W_c received **do**
   - Verify $H(W_c)$ matches on-chain hash
   - **if** valid **then** store W_c in transient store
7. **if** MissingSet $\neq \emptyset$ **then**
   - Broadcast pull request to gossip network
   - Retry with fixed-interval sleep
8. Proceed to commit (missing data tracked for background reconciliation)

5.3 Dynamic Membership and Reconciliation

Collection membership may change over the lifetime of a network. When a new member is added to a collection's membership policy $\mathcal{M}$, the peer detects the configuration change during the commit of the block containing the policy update. At this point, all previously recorded *ineligible missing data* for that collection—entries the peer had tracked but was not authorized to fetch—are reclassified as *eligible missing data*. Additionally, the peer scans the committed blocks for any historical transactions that wrote to the collection and marks them as eligible missing data. This reclassification automatically triggers the background reconciliation service, which periodically pulls these historical private write-sets from existing collection members using the same hash-based verification as the reactive retrieval protocol, ensuring the new member eventually obtains the complete private data history for the collection.

When a member is removed from a collection, the removed peer retains access only to data committed before its removal. The updated membership policy excludes it from both proactive dissemination and reactive retrieval eligibility checks, ensuring that future private data is not shared with the departing organization.

5.4 Consistency and Fault Tolerance

A critical property of the retrieval mechanism is its idempotency: since multiple peers may attempt to pull the same missing data

concurrently, the system utilizes the versioned hashes on the public ledger as unique identifiers, ensuring that nodes eventually converge on a consistent Private State even in the presence of network partitions or concurrent block commits.

Remark (Integrity Independence). *Data integrity is preserved even when peers attempt to provide fraudulent pre-images, because the requester validates every incoming data piece against the on-chain hash recorded during the consensus phase. This integrity guarantee operates independently of the CFT assumption described in §3: while the* ***liveness*** *of dissemination depends on honest peers following the protocol, the* ***correctness*** *of retrieved data is ensured solely by cryptographic hash verification. A single honest data source is sufficient for correctness.*

6 THREAT MODEL AND SECURITY ANALYSIS

Our design assumes a decentralized network of organizations where a subset of nodes may be faulty or malicious. We describe the trust assumptions for each entity and analyze the security implications when these assumptions are violated.

Peer Nodes. We assume a crash-fault tolerant (CFT) model for authorized peers. A malicious peer may attempt to (a) leak private data it is authorized to see, or (b) refuse to disseminate data, but a single compromised peer cannot manipulate the ledger state because the integrity of the public ledger is protected by the endorsement policy, which requires signatures from multiple organizations before a transaction is considered valid. Data manipulation is only possible if a majority of the endorsing organizations collude. *Correctness* is preserved as long as the endorsement policy holds—that is, a sufficient number of endorsing organizations remain honest. If a majority of the endorsing organizations collude, they can endorse fraudulent transactions with manipulated hashes, compromising both the private data and the on-chain evidence. Data *confidentiality* within a collection is bounded by the weakest member—a compromised member can exfiltrate any private data it is authorized to access. **This is an inherent limitation of any access-control-based (as opposed to cryptographic) privacy mechanism**: the system ensures that unauthorized parties never receive private data, but it cannot prevent an authorized party from misusing data it legitimately possesses. Organizations mitigate this risk through legal agreements, hardware security modules, and operational controls.

Ordering Service. The ordering service is a decentralized cluster operated by consortium organizations—potentially the same organizations that run peers. Its role is strictly limited to establishing the total order of transactions; it never sees private data payloads, as only public and hashed read-write sets are submitted for ordering. Standard BFT security assumptions apply: as long as fewer than one-third of the ordering nodes are Byzantine, the ordering service guarantees liveness and correct ordering.

Clients. Clients may be malicious, attempting to submit inconsistent private data to different endorsers or forge simulation results. If a client sends different private data to different endorsers, each endorser will compute a different hash of the private data and include it in its signed endorsement response. Since the endorsement policy requires matching responses from multiple endorsers, the inconsistent hashes will cause the endorsements to be incompatible, and the transaction will be rejected during validation. A malicious client may also simulate many transactions but never submit them to the ordering service, leading to the accumulation of orphan write-sets in the transient store. To mitigate this, we introduce a retention window Δ: the transient store automatically purges any entries where the current ledger height exceeds the simulation height by more than Δ. Receiving peers also reject disseminated write-sets outside this window. A second attack vector involves fabricated simulations lacking sufficient endorsements; to counter this, peers perform endorsement policy validation *before* attempting to pull any missing pre-images, skipping invalid transactions entirely.

Network. We assume a partially synchronous network where partitions are possible but eventually resolve, allowing for asynchronous data reconciliation. During a partition, private data dissemination may be delayed but the system does not compromise correctness—peers that cannot obtain private data before commit track it as missing and reconcile asynchronously once connectivity is restored.

Metadata Leakage. While private data collections protect the content of private data, they do not hide the *metadata* of private transactions. Since the hashed read-write sets recorded on the public ledger reference specific collections, and collection membership policies are visible to all network participants, any peer can infer which organizations are involved in a private transaction—even without access to the underlying data. For example, in the food supply chain scenario, a competitor observing that a transaction wrote to a collection shared between a specific manufacturer and supplier could infer the existence of a business relationship between them, even though the terms of the deal remain hidden. Addressing this metadata leakage would require techniques such as zero-knowledge proofs or oblivious dissemination, which we identify as a direction for future work.

7 IMPLEMENTATION

We implemented the proposed private data collection mechanism within Hyperledger Fabric [1], an open-source permissioned blockchain platform. The core mechanism was introduced in Fabric v1.1, became production-ready in v1.2 with dynamic membership and CouchDB support, gained automatic reconciliation in v1.4, added implicit per-organization collections and collection-level endorsement policies in v2.0, and introduced on-demand programmatic purging via the `PurgePrivateData` API in v2.5. The implementation maps the generic architectural components described in §4 to the peer's internal services.

Fabric Architecture Overview. Fabric implements the execute-order-validate-commit paradigm described in §3 (termed *simulate-order-validate-commit* in Fabric, as the execution phase is speculative). Communication between clients, peers, and the ordering service [30] uses gRPC [16]. Within a peer, the key components include the *endorser* (chaincode simulation), the *gossip* layer (block dissemination), endorsement policy and serializability validators, and a *committer* that persists valid transactions to the *block store*, *state DB*, and *history DB*. Figure 4 shows how the private data collection components (green) extend this existing architecture (white).

Storage Architecture. The peer's internal architecture, depicted in Figure 4, was extended to support the dual-layered storage strategy. All private data stores use LevelDB as the underlying engine. The

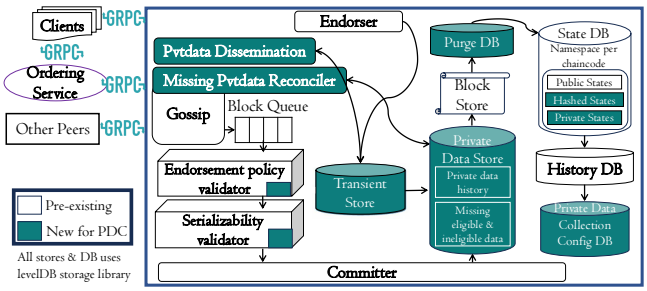

Figure 4: Peer architecture with private data collection components. Pre-existing components (white) form the core transaction pipeline; new components (green) implement the private data collection mechanism.

implementation introduces four logical stores: (i) a *transient store* for uncommitted write-sets awaiting dissemination and commit; (ii) a *private data store* maintaining the complete history of committed private data for retrieval and reconciliation; (iii) an *eligible missing data* tracker for private data the peer is authorized to access but has not yet received; and (iv) an *ineligible missing data* tracker for collections the peer is not currently a member of, which are reclassified as eligible upon membership changes.

Partitioned State Management. Partitioned state management is implemented by extending Fabric's existing State DB with namespace-based isolation. Each chaincode's public, hashed, and private state are stored in the same database but distinguished by internal key prefixes, ensuring that the peer's data management layer enforces strict access boundaries—a smart contract cannot directly access the hashed or private namespaces of other chaincodes.

Gossip Protocol Extensions. The dissemination and retrieval protocols described in §5 were integrated into Fabric's gossip component via new message types for both the proactive push of private write-sets during simulation (Algorithm 1) and the reactive pull during commit (Algorithm 2). The implementation includes a pull-retry mechanism with a fixed 1-second sleep interval, bounded by a configurable time threshold, to handle transient network failures without overwhelming the network.

Collection Configuration and Membership. Collection policies—corresponding to $C = (id, \mathcal{M}, \mathcal{E}, n_{dissem}, n_{required}, \tau)$ from §3—are stored as part of the chaincode definition in the channel's configuration. Policy updates are handled through the standard configuration update process; when a new organization is added, the peer's `reconciliation` service detects the membership change and initiates the retroactive reconciliation described in §5.3, pulling historical data for which the peer is now authorized.

Lifecycle and Purging. The system supports two complementary purging mechanisms. A background retention process, implemented as a scheduled task within the `PvtDataStore`, automatically purges expired private data based on the collection's block-to-live parameter τ. The purging interval is configurable (defaulting to every 100 blocks) and operates using a separate database iterator with small, asynchronous batches to avoid impacting transaction throughput. Additionally, the `PurgePrivateData` API (Table 2) enables application-initiated purging, allowing smart contracts to explicitly remove specific entries on demand. This is implemented via purge markers: when a purge is committed, the system records a marker that prevents the purged data from being returned in subsequent queries or reconciled from other peers, supporting compliance with "right to forget" regulations.

Validation and Commit. We extended the peer's validation component to check both chaincode-level endorsement policies and collection-level access policies before committing a transaction. For blind serializability validation, the implementation performs OCC checks by querying the `$$h` namespace in the State DB, allowing even nodes without the private data to verify that the hashed read-write set is consistent with the current ledger state. A key challenge was ensuring atomicity of updates across the public State DB and the `PvtDataStore`, which are separate storage components. We addressed this with a multi-stage commit: the public state is updated only after the private data is successfully persisted, preventing inconsistent states where a public hash is recorded but the corresponding private data is lost. Handling blind validation also required refactoring the OCC engine to support version lookups across different namespace prefixes, ensuring that version increments in the hashed space are correctly tracked alongside the public state.

Implementation Scope. The private data collection feature was implemented in Go and comprises approximately 12,000 lines of production code across the peer's endorser, committer, gossip, and state management subsystems—with the largest components being private data storage (~4,000 lines), gossip-based dissemination and retrieval (~3,300 lines), and the privacy-enabled state database (~1,700 lines). The feature includes approximately 21,000 lines of unit and integration tests (a 1.6:1 test-to-production ratio). The feature is maintained as part of the core Fabric codebase.

8 EVALUATION

TODO: Evaluation experiments to include:

(1) Throughput comparison: PDC transactions vs. standard transactions
(2) Latency breakdown across the four phases (simulation, ordering, validation, commit)
(3) Scalability with number of collections, organizations, and private data sizes
(4) Gossip overhead: network bandwidth for dissemination and retrieval
(5) Impact of block-to-live (purging) on performance
(6) Comparison with channel-based isolation (the main alternative)
(7) Failure/recovery scenarios and their performance impact
(8) Addition of new members to the collection and sync/fetch duration

9 REAL-WORLD IMPACT

Since its introduction in Hyperledger Fabric v1.2, the private data collection mechanism has seen significant adoption across diverse industries. In mining and metals, MineHub [21] explicitly credits PDCs for enabling their supply-chain platform to scale to thousands of companies, replacing the coarse-grained channel model

that could not support their dynamic data-sharing requirements. In food safety, the IBM Food Trust—deployed at Walmart and other major retailers [19]—leverages PDCs to protect supplier-specific data such as pricing and proprietary process steps while maintaining end-to-end traceability. In global trade, GSBN (Global Shipping Business Network) [22] uses PDCs for private trade documents such as Bills of Lading across its permissioned shipping network. In pharmaceuticals, the MediLedger project [26] combines PDCs with zero-knowledge proofs to achieve even higher levels of privacy for drug tracking under the U.S. Drug Supply Chain Security Act (DSCSA), as validated by an FDA pilot program. In aerospace, Honeywell's GoDirect Trade marketplace [18] uses channels and private data collections to secure transactions in its B2B marketplace for used aircraft parts, recording $1 million in sales in its first ten weeks. In financial services, Oracle's Blockchain Platform [31] uses Fabric's private data collections to protect account attributes in its wholesale Central Bank Digital Currency (CBDC) solution, storing bank account numbers only in the private data of participating organizations while replacing them with one-way hashes in the shared ledger. These deployments demonstrate that private data collections address a genuine enterprise need and that the design choices described in this paper—particularly the balance between on-chain verifiability and off-chain confidentiality—translate effectively to production environments.

10 RELATED WORK

Channels and Sub-Networks. Hyperledger Fabric's *channels* [1] provide data isolation via independent blockchain instances, each with its own ledger and consensus. While effective, channels are coarse-grained: each data-sharing relationship may require a separate channel, leading to operational complexity. Our private data collections provide fine-grained privacy within a single channel.

Private Transactions via Off-Chain Managers. Quorum [27] and Hyperledger Besu [20] both implement a hash-on-chain / data-off-chain model using an external transaction manager (Tessera). However, both have static privacy group membership, require all participants to be online, and offer limited interoperability between private and public state within a single transaction. Our mechanism supports dynamic membership with retroactive reconciliation, tolerates offline peers via gossip, and enables atomic cross-state reads and writes. R3 Corda [4] eliminates the global ledger entirely, sharing transactions only on a need-to-know basis. This introduces a *backchain resolution* problem: to verify that an input state is valid, a recipient must walk the chain of prior transactions that produced it, and each step in this chain may involve data from unrelated parties who never consented to its disclosure. The result is that verifying provenance can inadvertently expose historical data to new counterparties. Furthermore, the absence of a global ledger precludes network-wide queries and the tamper-evident audit trail that our on-chain hashes provide.

Cryptographic Privacy. Zero-knowledge proof systems provide strong privacy guarantees: Zerocash [2] uses zk-SNARKs [3] for fully anonymous payments; Hawk [23] extends this to smart contracts but requires a trusted manager; Zether [5] enables confidential token transfers via ElGamal encryption and range proofs. All incur significant computational cost (seconds per proof), making them unsuitable for high-throughput enterprise workloads. Our work is complementary—MediLedger [26] demonstrates combining PDCs with ZKPs in practice. Confidential Transactions [25], based on Pedersen commitments [32], hide amounts via homomorphic commitments but not sender/receiver identities. CryptoNote [35] (Monero) combines ring signatures and stealth addresses for anonymity. These address permissionless cryptocurrency privacy—a fundamentally different problem from selective data access in permissioned networks.

TEEs, MPC, and Homomorphic Encryption. TEE-based systems such as Ekiden [7] and CCF [34] execute smart contracts inside hardware enclaves (e.g., Intel SGX), but introduce hardware dependencies and side-channel vulnerabilities. MPC platforms like Enigma [37] enable joint computation over private inputs, but performance overhead led the project to pivot to TEEs. Fully homomorphic encryption [15] allows computation on encrypted data but incurs 10^4–$10^6\times$ overhead. Our approach sidesteps these costs: authorized peers process private data in plaintext while the blockchain stores only hashes, requiring no specialized hardware or expensive cryptographic primitives.

Database-Layer Privacy. CryptDB [33] executes SQL over encrypted data with practical overhead (15–26%) but progressively leaks information. Differential privacy [8] provides statistical guarantees against individual record disclosure. These are orthogonal to our work: PDCs control *who* accesses data at the replication layer, while database techniques control *what* is learned by authorized processors.

Positioning. Our mechanism uniquely provides fine-grained privacy through architectural means (selective replication, on-chain hashes, blind validation) rather than expensive cryptographic primitives. Unlike channels, it operates within a single ledger; unlike Corda, it maintains a global audit trail; unlike ZKP/HE approaches, it imposes minimal overhead; and unlike TEEs, it requires no specialized hardware.

11 CONCLUSION

We presented *private data collections*, a mechanism for managing private data on permissioned blockchains that stores private data off-chain on authorized peers while recording only its cryptographic hash on the replicated ledger. The design introduces a transient store, hashed read-write sets for blind validation, and a gossip-based dissemination protocol that together provide fine-grained privacy, transactional atomicity, integrity, and auditability. Our implementation in Hyperledger Fabric, merged into the codebase and production-ready since release v1.2, with continued enhancements through v2.5, demonstrates that private data collections can be integrated into an existing blockchain platform with minimal overhead, achieving throughput and latency comparable to standard transactions while significantly reducing the storage and management burden of channel-based isolation.

REFERENCES

[1] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, S. Muralidharan, C. Murthy, B. Nguyen, M. Sethi, G. Singh, K. Smith, A. Sorniotti, C. Stathakopoulou, M. Vukolić, S. W. Cocco, and J. Yellick. Hyperledger fabric: A distributed operating system for permissioned blockchains. In *Proceedings of the Thirteenth*

EuroSys Conference, EuroSys '18, pages 30:1–30:15, New York, NY, USA, 2018. ACM.
[2] E. Ben-Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza. Zerocash: Decentralized anonymous payments from Bitcoin. In *Proceedings of the 2014 IEEE Symposium on Security and Privacy (SP)*, pages 459–474. IEEE, 2014.
[3] E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza. Succinct non-interactive zero knowledge for a von neumann architecture. In *Proceedings of the 23rd USENIX Conference on Security Symposium*, SEC'14, page 781–796, USA, 2014. USENIX Association.
[4] R. G. Brown, J. Carlyle, I. Grigg, and M. Hearn. Corda: An introduction. R3 Technical White Paper, 2016.
[5] B. Bünz, S. Agrawal, M. Zamani, and D. Boneh. Zether: Towards privacy in a smart contract world. *IACR Cryptology ePrint Archive*, 2019:191, 2019.
[6] V. Buterin. Ethereum: A next-generation smart contract and decentralized application platform, 2014. Accessed: July 31, 2019.
[7] R. Cheng, F. Zhang, J. Kos, W. He, N. Hynes, N. Johnson, A. Juels, A. Miller, and D. Song. Ekiden: A platform for confidentiality-preserving, trustworthy, and performant smart contracts. In *Proceedings of the IEEE European Symposium on Security and Privacy*, EuroS&P '19, pages 185–200. IEEE, 2019.
[8] C. Dwork and A. Roth. The algorithmic foundations of differential privacy. *Foundations and Trends in Theoretical Computer Science*, 9(3–4):211–407, 2014.
[9] European Commission. Food safety policy. https://ec.europa.eu/food/horizontal-topics/general-food-law_en, 2023.
[10] European Parliament and Council of the European Union. General data protection regulation (GDPR) – regulation (EU) 2016/679, 2016.
[11] Food and Agriculture Organization of the United Nations. Food traceability guidance. http://www.fao.org/3/a-i7665e.pdf, 2017.
[12] Food Standards Australia New Zealand. Food standards Australia New Zealand. https://www.foodstandards.gov.au/, 2023.
[13] P. Fusion. New data reveals average american sees 18.7 different doctors in a lifetime, 2014.
[14] J. S. A. G. C. N. Gaur. *Blockchain for Business*. Addison-Wesley Professional, paperback edition, 2019.
[15] C. Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing*, STOC '09, pages 169–178, New York, NY, USA, 2009. ACM.
[16] Google. gRPC: A high-performance, open-source universal RPC framework. https://grpc.io/, 2023.
[17] HHS. Hipaa privacy rule.
[18] Hyperledger Foundation. Case study: Honeywell aerospace creates online parts marketplace with Hyperledger Fabric. https://www.lfdecentralizedtrust.org/case-studies/honeywell-case-study, 2019.
[19] Hyperledger Foundation. How Walmart brought unprecedented transparency to the food supply chain with Hyperledger Fabric. https://www.hyperledger.org/wp-content/uploads/2019/02/Hyperledger_CaseStudy_Walmart_Printable_V4.pdf, 2019.
[20] Hyperledger Foundation. Hyperledger Besu. https://www.hyperledger.org/use/besu, 2020.
[21] Hyperledger Foundation. Case study: Managing the metal and mining industry's supply chain. https://www.hyperledger.org/case-studies/minehub, 2021.
[22] Hyperledger Foundation. GSBN simplifies global trade with Hyperledger Fabric. https://www.hyperledger.org/case-studies/gsbn, 2022.
[23] A. Kosba, A. Miller, E. Shi, Z. Wen, and C. Papamanthou. Hawk: The blockchain model of cryptography and privacy-preserving smart contracts. In *2016 IEEE Symposium on Security and Privacy (SP)*, pages 839–858, 2016.
[24] H. T. Kung and J. T. Robinson. On optimistic methods for concurrency control. *ACM Trans. Database Syst.*, 6(2):213–226, June 1981.
[25] G. Maxwell. Confidential transactions, 2015.
[26] MediLedger. DSCSA pilot project report. https://www.mediledger.com/fda-pilot-report, 2020.
[27] J. Morgan-Chase. Quorum private transactions.
[28] S. Nathan, C. Govindarajan, A. Saraf, M. Sethi, and P. Jayachandran. Blockchain meets database: Design and implementation of a blockchain relational database. *Proc. VLDB Endow.*, 12(11):1539–1552, July 2019.
[29] National Center for Biotechnology Information. Escherichia coli. https://www.ncbi.nlm.nih.gov/books/NBK507845/, 2023.
[30] D. Ongaro and J. Ousterhout. In search of an understandable consensus algorithm. In *Proceedings of the 2014 USENIX Conference on USENIX Annual Technical Conference*, USENIX ATC'14, pages 305–320, Berkeley, CA, USA, 2014. USENIX Association.
[31] Oracle. Robust, scalable and efficient system for confidential token transactions: Oracle's breakthrough on Hyperledger Fabric. https://blogs.oracle.com/blockchain/robust-scalable-and-efficient-system-for-confidential-tokenized-transactions-oracles-breakthrough-on-hyperledger-fabric, 2024.
[32] T. P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In J. Feigenbaum, editor, *Advances in Cryptology — CRYPTO '91*, pages 129–140, Berlin, Heidelberg, 1992. Springer Berlin Heidelberg.
[33] R. A. Popa, C. M. S. Redfield, N. Zeldovich, and H. Balakrishnan. Cryptdb: Protecting confidentiality with encrypted query processing. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*, SOSP '11, pages 85–100, New York, NY, USA, 2011. ACM.
[34] M. Russinovich, E. Ashton, C. Avanessians, M. Castro, A. Chamayou, S. Clebsch, M. Costa, C. Fournet, M. Kerner, S. Krishna, J. Maffre, T. Shermer, and O. Roy. CCF: A framework for building confidential verifiable replicated services. Technical Report MSR-TR-2019-16, Microsoft Research, 2019.
[35] N. van Saberhagen. CryptoNote v 2.0. Whitepaper, 2013.
[36] Walmart Inc. Walmart and Sam's Club to require real-time, end-to-end food traceability with blockchain. https://corporate.walmart.com/newsroom/2018/09/24/in-wake-of-romaine-e-coli-scare-walmart-deploys-blockchain-to-track-leafy-greens, 2018.
[37] G. Zyskind, O. Nathan, and A. Pentland. Enigma: Decentralized computation platform with guaranteed privacy. arXiv:1506.03471, 2015.